ML/DL 베이스라인 모델 구축

'일단 돌아가는 기준선'을 먼저 만드는 법 · 학습 정리 노트

1. 베이스라인이란?

베이스라인(baseline)은 **가장 단순하고 빠르게 만들 수 있는 기준 모델**입니다. 이후에 만드는 모든 복잡한 모델은 "이 기준선보다 얼마나 나은가?"로 평가됩니다. 성능 자체보다 **비교의 기준점**을 만드는 것이 목적입니다.

- **문제 난이도 파악** – 단순 모델만으로 이미 높은 점수가 나오면 문제가 쉽거나 데이터에 강한 신호가 있다는 뜻입니다.
- **파이프라인 검증** – 데이터 로딩·전처리·평가 코드의 버그를 복잡한 모델을 얹기 전에 잡을 수 있습니다.
- **개선 효과의 정량화** – "모델을 바꿨더니 +3%p"처럼 비교할 대상이 생깁니다.
- **복잡도 정당화** – 무거운 모델이 단순 모델보다 크게 낫지 않다면 운영 비용만 늘어납니다.

2. 구축 절차 (6단계)

단계	할 일	핵심 포인트
① **문제·지표 정의**	분류/회귀/랭킹 등 문제 유형과 평가 지표 결정	비즈니스 목표와 맞는 지표 선택 (불균형이면 Accuracy 대신 F1, PR-AUC 등)
② **데이터 분할**	train / validation / test 분리 (또는 교차검증)	**분할을 먼저, 전처리는 나중에** – 테스트셋은 마지막까지 봉인
③ **더미 베이스라인**	최빈값·평균값·직전 값 같은 무(無)학습 예측	"아무것도 안 하는 모델"의 점수 = 반드시 넘어야 할 최저선
④ **단순 모델**	선형/로지스틱 회귀, 결정트리, 트리 앙상블 등	기본 하이퍼파라미터로 시작, 튜닝은 나중에
⑤ **검증·오류 분석**	교차검증, 혼동행렬, 틀린 샘플 직접 확인	점수 평균뿐 아니라 **표준편차**(분산)도 기록
⑥ **기록·재현성**	시드, 데이터/코드 버전, 지표를 표로 기록	이후 실험과 같은 조건으로 비교할 수 있어야 함

3. 데이터 유형별 베이스라인 사다리

'더미 → 단순 모델 → 강한 모델' 순서로 한 칸씩 올라가며 매번 점수를 비교합니다.

데이터	더미 / 최저선	단순 베이스라인	강한 베이스라인
정형(tabular)	최빈값 / 평균값	로지스틱·선형 회귀(+정규화)	Random Forest, LightGBM, XGBoost
텍스트	최빈 클래스	TF-IDF + 로지스틱 회귀 / 선형 SVM	사전학습 언어모델(BERT 계열) 파인튜닝
이미지	최빈 클래스	사전학습 CNN 특징 + 선형 분류기	사전학습 모델(ResNet 등) 파인튜닝
시계열	직전 값(naive), 계절 naive	이동평균, ETS/ARIMA	lag·rolling 피처 + GBM
추천	인기도(Popularity) 기반	아이템/유저 기반 협업 필터링	행렬분해(MF), 두 타워 모델

> **핵심 원칙** · 정형 데이터에서는 딥러닝보다 **트리 기반 모델(GBM)**이 강한 베이스라인인 경우가 많고, 텍스트·이미지는 **사전학습 모델 + 전이학습**이 사실상 기본 베이스라인입니다. "처음부터 학습한 거대 모델"은 베이스라인이 아니라 다음 단계의 실험입니다.

4. 딥러닝 베이스라인을 만들 때의 원칙

딥러닝은 버그가 있어도 에러 없이 '그럴듯하게' 학습되기 때문에, **작게 시작해서 한 가지씩 검증**하는 습관이 중요합니다.

- **가장 단순한 구조로 시작** – 작은 MLP/CNN 또는 사전학습 모델의 마지막 층만 교체해 학습합니다.
- **기본값 활용** – Adam 옵티마이저, 학습률 약 1e-3(처음부터 학습) / 1e-5~5e-5(파인튜닝), 적당한 배치 크기로 시작합니다.
- **초기 손실 확인** – C-클래스 분류에서 초기 손실이 대략 ln(C) 근처인지 확인해 출력층·손실함수 오류를 잡습니다.
- **소규모 과적합 테스트** – 데이터 일부(수십~수백 개)에 과적합이 되는지 확인합니다. 안 된다면 모델/학습 코드에 버그가 있다는 신호입니다.
- **한 번에 하나만 변경** – 구조, 학습률, 증강을 동시에 바꾸면 무엇이 효과였는지 알 수 없습니다.
- **시드 고정과 여러 번 실행** – 시드가 다른 2~3회 결과의 평균으로 비교해야 우연한 차이를 걸러낼 수 있습니다.

5. 흔한 실수와 평가 시 주의점

실수	왜 문제인가	대처
데이터 누수(leakage)	스케일링·인코딩을 전체 데이터에 fit하면 검증 정보가 학습에 새어 들어가 점수가 부풀려짐	분할 후 train에만 fit, **Pipeline** 사용
불균형 데이터에서 Accuracy	다수 클래스만 찍어도 높은 점수가 나옴	F1, PR-AUC, 클래스별 재현율 확인
시계열 랜덤 분할	미래 데이터로 과거를 예측하는 셈이 됨	시간 순서대로 분할(walk-forward)
테스트셋 반복 사용	테스트셋에 모델을 맞추게 되어 일반화 성능을 과대평가	검증셋/CV로 선택, 테스트는 마지막 1회
단일 실행 결과 신뢰	시드·분할에 따른 변동을 개선으로 착각	CV 평균±표준편차, 여러 시드

6. 코드 예시 – scikit-learn 베이스라인 비교

```
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import cross_val_score, StratifiedKFold

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

models = {
    "dummy":  DummyClassifier(strategy="most_frequent"),
    "logreg": make_pipeline(StandardScaler(),        # CV 안에서만 fit -> 누수 방지
                            LogisticRegression(max_iter=1000)),
}
for name, m in models.items():
    s = cross_val_score(m, X, y, cv=cv, scoring="f1_macro")
    print(f"{name}: {s.mean():.3f} +/- {s.std():.3f}")
```

7. 베이스라인 이후: 개선 판단 기준

- 개선폭이 CV 표준편차보다 작다면 **노이즈일 가능성**이 큽니다. 같은 분할·같은 지표로 비교하세요.
- 성능 향상 대비 **학습·추론 비용, 해석 가능성, 유지보수 부담**도 함께 따져 채택 여부를 결정합니다.
- 오류 분석에서 확인한 약점(특정 클래스, 구간, 피처)을 근거로 다음 실험을 고릅니다.

베이스라인 체크리스트

☐ 지표를 정했고, 더미 모델 점수를 기록했다
☐ 분할을 먼저 하고, 전처리는 train에만 fit했다 (누수 없음)
☐ 단순 모델을 기본 설정으로 먼저 돌려 점수를 남겼다
☐ 시드·데이터/코드 버전·결과를 기록해 재현할 수 있다
☐ 틀린 샘플을 직접 확인하고, 다음 실험은 한 번에 하나만 바꾼다